Musil

Scripting Language for Sound And Music Computing

User Manual v0.51

Musil is a small, expressive scripting language designed as a control and computation layer for algorithmic composition, signal processing, and music technology research. It provides first-class numeric vectors, heterogeneous arrays, first-class procedures, file I/O, and a clean extension API for connecting to C++ libraries.

(c) 2026 by Carmine-Emanuele Cella

Contents

1 Overview **3**

2 Types and Values **3**
2.1 Numbers (scalars) . . . 4
2.2 Numeric vectors . . . 4
2.3 Arrays . . . 4
2.4 Strings . . . 5
2.5 Procs . . . 5

3 Variables and Expressions **5**
3.1 Variable declaration and assignment . . . 5
3.2 Operator precedence . . . 5
3.3 Arithmetic on vectors and scalars . . . 6
3.4 Comparison operators . . . 6
3.5 Logical operators . . . 6

4 Control Flow **6**
4.1 If / else if / else . . . 6
4.2 While . . . 6
4.3 For / in . . . 7
4.4 Return . . . 7

5 Procedures **7**
5.1 Named procedures . . . 7
5.2 Anonymous proc literals . . . 7
5.3 Calling procs by value . . . 7
5.4 Higher-order patterns . . . 8
5.5 Recursion . . . 8

6 Vectors — Numeric Arrays **8**
6.1 Creation . . . 8
6.2 Element-wise arithmetic . . . 8
6.3 Math functions (all element-wise) . . . 8
6.4 Reductions . . . 9
6.5 Indexing . . . 9
6.6 Iteration . . . 9
6.7 Conversion . . . 9

7 Arrays — Heterogeneous Lists **9**
7.1 Creation . . . 9
7.2 Indexing and assignment . . . 9
7.3 Mutation . . . 10
7.4 Non-mutating operations . . . 10
7.5 Reference semantics and copy . . . 10

8 Strings **10**

9 File I/O and Modules **11**
9.1 File I/O . . . 11
9.2 Loading modules . . . 11

9.3 Eval and exec . . . 11

10 Built-in Functions Reference **11**
10.1 Math . . . 11
10.2 Vector constructors and reductions . . . 12
10.3 String builtins . . . 12
10.4 Array builtins . . . 12
10.5 Type and conversion . . . 13
10.6 File I/O and modules . . . 13
10.7 Higher-order and control . . . 13

11 Libraries **14**
11.1 Standard library . . . 14
11.1.1 Constants . . . 14
11.1.2 Core math . . . 14
11.1.3 Number formatting . . . 15
11.1.4 String utilities . . . 15
11.1.5 Vector statistics . . . 15
11.1.6 Array algorithms . . . 15
11.1.7 Higher-order functions . . . 16
11.1.8 Testing framework . . . 16
11.2 System library . . . 16
11.3 Signals library . . . 17
11.4 Scientific library . . . 18
11.5 Plotting library . . . 18

12 Extending Musil from C++ **19**
12.1 Including Musil . . . 19
12.2 Registering a builtin . . . 19
12.3 Injecting global variables . . . 19
12.4 Value types in C++ . . . 19

13 The Musil IDE **19**
13.1 Overview . . . 20
13.2 Basic Features . . . 20
13.3 File Management . . . 20
13.4 Libraries and Installation . . . 20
13.5 Notes . . . 21

14 Quick Reference **21**

1 Overview

Musil is a dynamically-typed, interpreted scripting language implemented as a single C++ header (`musil.h`). It is designed for use as a frontend for C++ tools in music technology, orchestration, and signal processing.

Key characteristics

- **Four value types:** number (scalar), vector (numeric array), string, array (heterogeneous), and proc (first-class function).
- **Numeric vectors** are `std::valarray<double>` under the hood; all arithmetic and math functions apply element-wise with scalar broadcasting.
- **Heterogeneous arrays** hold any mix of values; they have reference semantics (assignment shares the data).
- **First-class procedures** can be stored in variables, passed as arguments, and stored in arrays.
- **Modular:** `load("file.mu")` executes another file in the current environment; the `~/.musil/` directory is on the search path.
- **Extensible:** register C++ functions as Musil builtins with one line.

Building and running

To compile using CMake, from the *root* folder type:

```
mkdir build
cd build
cmake ..
make
```

You can also compile the code manually, from the src folder, with:

```
g++ -std=c++17 -O2 -o musil musil.cpp    # compile the interpreter
```

To run the interpreter, call:

```
./musil script.mu                          # run a file
./musil lib.mu script.mu                   # run multiple files, shared
    state
./musil                                    # start interactive REPL
```

Files loaded with `load()` search first in the current directory, then in `~/.musil/`.

2 Types and Values

Musil has five types. Use `type(x)` to inspect any value at runtime.

Type	`type()` result	**C++ representation**	**Semantics**
Scalar number	`"number"`	`valarray<double>` size 1	value (copies on assign)
Numeric vector	`"vector"`	`valarray<double>` size >1	value (copies on assign)
String	`"string"`	`std::string`	value (copies on assign)
Array	`"array"`	`shared_ptr<MusilArray>`	reference (shares on assign)
Proc	`"proc"`	`shared_ptr<Proc>`	reference (shares on assign)

2.1 Numbers (scalars)

Every numeric literal creates a 1-element vector, which behaves as an ordinary number in all arithmetic and comparisons. The internal representation is `std::valarray<double>`; scalar operations compile to single-element valarray arithmetic and are as fast as raw `double`.

```
var x = 42
var y = 3.14
var z = .5          # leading dot supported
var tiny = 1e-10    # scientific notation supported
var big  = 6.674e-11
```

Musil has no integer type. All numbers are IEEE 754 doubles. Use `floor`, `ceil`, and `mod` when integer arithmetic is needed.

2.2 Numeric vectors

A vector is a `std::valarray<double>` with more than one element. It is created with the `vec`, `linspace`, `zeros`, `ones`, or `range-to-to_vec` functions. All arithmetic operators and all math builtins (`sin`, `cos`, `sqrt`, etc.) apply element-wise.

```
var v = vec(1, 2, 3, 4, 5)      # from individual scalars
var w = linspace(0, 1, 5)       # 5 points evenly spaced in [0, 1]
var z = zeros(4)                # <0 0 0 0>
var o = ones(3)                 # <1 1 1>

print v + 10         # <11 12 13 14 15>   scalar broadcast
print v * v          # <1 4 9 16 25>      element-wise
print sin(w * PI)    # element-wise sin
```

Vectors print as `<1 2 3>` with angle brackets, distinguishing them from arrays which print as `[1, 2, 3]`.

Value semantics: assigning a vector copies all its data. Mutating the copy does not affect the original.

```
var a = vec(1, 2, 3)
var b = a            # b is an independent copy
b[0] = 99
print a[0]           # a is unchanged
```

2.3 Arrays

An array is a heterogeneous, mutable, ordered list of values. Elements may be numbers, strings, other arrays, or procs. The backing store is a `shared_ptr<vector<Value>>`.

```
var a = [1, "two", [3, 4], proc(x){ return x*2 }]
print a[0]           # 1
print a[1]           # two
print a[2][1]        # 4      -- nested indexing
print a[3](5)        # 10     -- calling a stored proc
```

Reference semantics: assigning an array copies the *pointer*, not the data. Both variables refer to the same underlying list. Use `copy(a)` to obtain an independent shallow copy.

```
var x = [1, 2, 3]
var y = x            # y and x share the same list
y[0] = 99
```

```
print x[0]            # x sees the change

var z = copy(x)       # z is a new, independent copy
z[0] = 0
print x[0]            # x is unaffected
```

`copy()` is a *shallow* copy: inner arrays are still shared. `range(lo, hi)` also produces an array (suitable for `for`/`in` loops). For numeric computation, prefer `linspace` which produces a vector.

2.4 Strings

Strings are immutable. The `+` operator concatenates and coerces numbers:

```
var s = "score: " + 99        # "score: 99"
var t = "ha" + "ha"           # "haha"
```

Escape sequences: `\n` (newline), `\t` (tab), `\\` (backslash), `\"` (double quote).

2.5 Procs

Procs are first-class values. A named `proc` declaration stores the proc in the global proc table; an anonymous `proc` literal creates a `shared_ptr<Proc>` value.

```
proc square (n) { return n * n }      # named proc

var double = proc (x) { return x * 2 }    # anonymous, stored in variable
print double(7)     # 14
```

Note: Procs are *not* closures. They see the global environment at call time, not the local variables of the scope in which they were defined.

3 Variables and Expressions

3.1 Variable declaration and assignment

```
var name = expression       # declare and initialise
name = expression           # assign to existing variable
```

`var` inside a proc creates a local variable. `var` at the top level creates a global. Plain assignment (`=` without `var`) searches locals first, then globals; creates a new binding if not found.

3.2 Operator precedence

Precedence	Operators	Associativity
1 (lowest)	`or`	left
2	`and`	left
3	`not`	right (unary)
4	`< > <= >= == !=`	non-associative
5	`+ -`	left
6	`* /`	left
7	`-` (unary)	right
8 (highest)	`[i]` subscript, `()` call	left

3.3 Arithmetic on vectors and scalars

All arithmetic operators are element-wise on vectors. When one operand is a scalar (1-element vector) and the other is a multi-element vector, the scalar is *broadcast* to match the vector size:

```
var v = vec(1, 2, 3)
print v + 10              # <11 12 13>     scalar broadcast
print v * v               # <1 4 9>        element-wise
print v + vec(10,20,30)   # <11 22 33>     vector + vector
```

Mismatched sizes (neither is 1) produce a runtime error.

3.4 Comparison operators

On scalars, comparisons return `1` (true) or `0` (false). On vectors, comparisons are element-wise and return a *mask vector* of 0s and 1s:

```
print 5 > 3               # 1
print vec(1,5,3) > 2      # <0 1 1>    mask
print all(vec(1,5,3) > 0)   # all elements > 0
print any(vec(1,5,3) > 4)   # some elements > 4
```

Use `all()` and `any()` to reduce a vector mask to a scalar for use in `if` and `while` conditions.

String equality uses `==` and `!=` only; ordering operators on strings produce a runtime error.

3.5 Logical operators

`and`, `or`, `not` always return a scalar `1` or `0`. Truthiness: any non-zero number, any non-empty string, any non-empty array, any proc is truthy; zero, empty string, and empty array are falsy.

4 Control Flow

4.1 If / else if / else

```
if (condition) {
    ...
} else if (other_condition) {
    ...
} else {
    ...
}
```

Parentheses around the condition are required. Any number of `else if` branches may appear; the final `else` is optional.

4.2 While

```
while (condition) {
    ...
    break        # exit the loop immediately
}
```

`break` exits the innermost `while` or `for` loop. There is no `continue`.

4.3 For / in

Iterates over arrays, numeric vectors, or strings:

```
# Over an array (range returns an array)
for (var i in range(0, 5)) {
    print i
}

# Over a numeric vector
for (var x in linspace(0, 1, 5)) {
    print x
}

# Over a string (yields one character per iteration)
for (var ch in "hello") {
    print ch
}
```

The loop variable is declared with `var`; it is visible within the loop body. `break` works inside `for`/`in`.

> **Note:** `range(lo, hi)` produces an *array*, while `linspace(lo, hi, n)` produces a *vector*. Use `range` for integer loops, `linspace` for mathematical sequences.

4.4 Return

`return expression` exits the current proc and passes the value to the caller. A proc that falls off the end without `return` returns `0`.

5 Procedures

5.1 Named procedures

```
proc name (param1, param2, ...) {
    var local = ...
    return value
}
```

Named procs are stored in a global proc table and are available everywhere after their declaration. Parameters are local variables; modifications to parameter *vectors* do not affect the caller (value semantics). Modifications to parameter *arrays do* affect the caller (reference semantics).

5.2 Anonymous proc literals

```
var f = proc (x) { return x * x }
print f(5)          # 25
print type(f)       # proc
```

Anonymous procs can be stored in variables, passed to other procs, and stored in arrays. They are first-class values.

5.3 Calling procs by value

When a variable holds a proc, calling it uses the same `name(args)` syntax:

```
var transforms = [
    proc (x) { return x + 1 },
    proc (x) { return x * 2 }
]
print transforms[0](5)    # 6
print transforms[1](5)    # 10
```

5.4 Higher-order patterns

```
proc map (a, f) {
    var out = []
    for (var x in a) { push(out, f(x)) }
    return out
}
var doubled = map([1,2,3,4], proc(x){ return x*2 })
```

The stdlib provides `map`, `filter`, and `reduce`. The `apply(f, args)` builtin calls a proc value or a named proc with an array of arguments.

5.5 Recursion

Recursive calls work normally; there is no tail-call optimisation, so very deep recursion will exhaust the C++ call stack.

```
proc fib (n) {
    if (n <= 1) { return n }
    return fib(n-1) + fib(n-2)
}
```

6 Vectors — Numeric Arrays

6.1 Creation

```
var v = vec(1, 2, 3, 4, 5)          # from scalar arguments
var w = linspace(0, PI, 512)        # 512 points, 0 to PI inclusive
var z = zeros(1024)                  # 1024 zeros
var o = ones(256)                    # 256 ones
```

6.2 Element-wise arithmetic

```
var v = vec(1, 2, 3)
print v + 10            # <11 12 13>
print v * v             # <1 4 9>
print -v                # <-1 -2 -3>
print pow(v, 2)         # <1 4 9>  (element-wise power)
```

6.3 Math functions (all element-wise)

`floor`, `ceil`, `abs`, `sqrt`, `sin`, `cos`, `tan`, `exp`, `log`, `log2` accept a single vector argument. `pow(v, e)` and `atan2(y, x)` accept two vector arguments (with broadcast).

```
var phase = linspace(0, TAU, 512)
var wave  = sin(phase)              # one call, 512 sine values
var env   = linspace(1, 0, 512)     # linear fade
var faded = wave * env              # element-wise product
```

6.4 Reductions

```
var v = vec(1, 2, 3, 4, 5)
print sum(v)          # 15
print all(v > 0)      # 1
print any(v > 4)      # 1
print len(v)          # 5
```

6.5 Indexing

Negative indices count from the end (`-1` is the last element).

```
var v = vec(10, 20, 30)
print v[0]      # 10
print v[-1]     # 30
v[1] = 99
print v         # <10 99 30>
```

6.6 Iteration

```
var total = 0
for (var x in vec(1, 2, 3, 4, 5)) {
    total = total + x
}
```

6.7 Conversion

`to_arr(v)` converts a vector to an array of numbers. `to_vec(a)` converts an array of numbers to a vector (fails if any element is non-numeric).

7 Arrays — Heterogeneous Lists

7.1 Creation

```
var a = [1, "two", [3, 4]]       # literal
var b = arr(5)                    # [0, 0, 0, 0, 0]
var c = arr(3, "x")               # ["x", "x", "x"]
var d = range(0, 5)               # [0, 1, 2, 3, 4]
var e = split("a,b,c", ",")       # ["a", "b", "c"]
```

7.2 Indexing and assignment

```
print a[0]           # first element
print a[-1]          # last element
a[1] = "TWO"         # mutates in place (visible through all aliases)

var matrix = [[1,2,3],[4,5,6]]
```

```
print matrix[1][2]   # 6
matrix[1][2] = 99    # nested assignment
```

7.3 Mutation

All mutation functions modify the array in place. Because arrays have reference semantics, mutations are visible through all aliases.

```
var a = [1, 2, 3]
push(a, 4)            # [1, 2, 3, 4]
push(a, 5, 6)         # variadic: [1, 2, 3, 4, 5, 6]
var x = pop(a)        # removes and returns 6
insert(a, 2, 99)      # insert before index 2: [1, 2, 99, 3, 4, 5]
var r = remove(a, 0)  # removes index 0, returns 1
```

7.4 Non-mutating operations

These return new arrays; the original is unchanged.

```
var s = slice(a, 1, 4)        # elements [1, 4)
var c = concat(a, b)          # all elements of a then b
var cp = copy(a)              # shallow copy
var sh = shuffle(a)           # randomly shuffled copy
```

7.5 Reference semantics and copy

```
var x = [1, 2, 3]
var y = x                # y aliases x
y[0] = 99
print x[0]               # shared mutation

var z = copy(x)          # break sharing
z[0] = 0
print x[0]               # x unaffected
```

Warning: Arrays passed to procs are passed by reference. A proc that calls `push`, `pop`, `insert`, or `remove` on a parameter will modify the caller's array. Call `copy(a)` inside the proc if you need a local version.

8 Strings

Strings are immutable value types. The + operator concatenates; if either operand is a number it is converted to string first.

```
var s = "Hello, " + "Musil!"
var t = "score: " + 99         # "score: 99"
print len(s)                   # 12
print upper(s)                 # "HELLO, MUSIL!"
print sub(s, 0, 5)             # "Hello"
print find(s, ",")             # 5
```

Strings can be iterated character by character with `for`/`in`.

9 File I/O and Modules

9.1 File I/O

```
write("/tmp/out.txt", "line one\nline two\n")     # create or overwrite
append("/tmp/out.txt", "line three\n")            # add to end
var content = read("/tmp/out.txt")                # whole file as string
```

9.2 Loading modules

```
load("stdlib.mu")           # search current dir, then ~/.musil/
load("/abs/path/lib.mu")    # absolute path
```

All procs and variables defined in the loaded file become globally available in the calling environment. Files are executed sequentially; circular loads will loop indefinitely.

9.3 Eval and exec

```
eval("var x = " + str(42))    # parse and execute a string as Musil code
eval("proc cube(n){ return n*n*n }")

var rc = exec("ls -l /tmp")   # run a shell command; returns exit code
```

10 Built-in Functions Reference

All built-in functions are always available without loading any library. Functions marked **[vec]** operate element-wise on vectors. Functions marked **[arr]** require an array argument.

10.1 Math

Function	Signature	Description
`floor` [vec]	`floor(x)`	Largest integer $\leq x$.
`ceil` [vec]	`ceil(x)`	Smallest integer $\geq x$.
`abs` [vec]	`abs(x)`	Absolute value.
`sqrt` [vec]	`sqrt(x)`	Square root.
`sin` [vec]	`sin(x)`	Sine (radians).
`cos` [vec]	`cos(x)`	Cosine (radians).
`tan` [vec]	`tan(x)`	Tangent (radians).
`exp` [vec]	`exp(x)`	e^x.
`log` [vec]	`log(x)`	Natural logarithm.
`log2` [vec]	`log2(x)`	Base-2 logarithm.
`pow` [vec]	`pow(base, exp)`	$\texttt{base}^{\texttt{exp}}$, element-wise with broadcast.
`atan2` [vec]	`atan2(y, x)`	Four-quadrant arctangent, element-wise.
`rand`	`rand()`	Pseudo-random number in $[0, 1)$.

10.2 Vector constructors and reductions

Function	Signature	Description
`vec`	`vec(a, b, ...)`	Create a vector from scalar arguments.
`linspace`	`linspace(lo, hi, n)`	n evenly-spaced values in [`lo`, `hi`] inclusive.
`zeros`	`zeros(n)`	Vector of n zeros.
`ones`	`ones(n)`	Vector of n ones.
`sum`	`sum(v)`	Sum of all elements (vectors only; use `total` for arrays).
`all`	`all(v)`	Returns 1 if every element is non-zero.
`any`	`any(v)`	Returns 1 if any element is non-zero.
`to_arr`	`to_arr(v)`	Convert vector to array of numbers.
`to_vec`	`to_vec(a)`	Convert array of numbers to vector.

10.3 String builtins

Function	Signature	Description
`len`	`len(x)`	Length of string, array, or vector.
`sub`	`sub(s, lo, hi)`	Substring `s[lo..hi)`.
`find`	`find(s, pattern)`	First occurrence of `pattern` in `s`, or -1.
`upper`	`upper(s)`	Uppercase copy.
`lower`	`lower(s)`	Lowercase copy.
`str`	`str(x)`	Convert any value to its string representation.
`num`	`num(s)`	Parse string to number.
`char`	`char(n)`	Character with ASCII code n.
`asc`	`asc(s)`	ASCII code of first character of `s`.
`split`	`split(s, delim)`	Split string on delimiter; returns array of strings.

10.4 Array builtins

Function	Signature	Description
`arr`	`arr()`	Empty array.
	`arr(n)`	Array of n zeros.
	`arr(n, val)`	Array of n copies of `val`.
`len`	`len(a)`	Number of elements.
`push`	`push(a, v1, v2, ...)`	Append one or more values; mutates in place. Returns `a`.
`pop`	`pop(a)`	Remove and return last element; mutates.
`insert`	`insert(a, i, val)`	Insert `val` before index `i`; mutates. Returns `a`.
`remove`	`remove(a, i)`	Remove element at index `i`; mutates. Returns removed element.
`slice`	`slice(a, lo, hi)`	New array with elements `[lo, hi)`.
`concat`	`concat(a, b)`	New array with all elements of `a` then `b`.
`copy`	`copy(a)`	Shallow copy; breaks reference sharing.
`range`	`range(lo, hi)`	Array of integers [`lo`, `hi`).

Function	Signature	Description
	`range(lo, hi, step)`	With custom step; negative step goes downward.
`shuffle`	`shuffle(a)`	New randomly shuffled copy. Original unchanged.
`join`	`join(a, sep)`	Concatenate elements as strings with separator. Returns string.
`keys`	`keys()`	Array of all current global variable names (debugging).

10.5 Type and conversion

Function	Signature	Description
`type`	`type(x)`	Returns `"number"`, `"vector"`, `"string"`, `"array"`, or `"proc"`.
`str`	`str(x)`	Any value to string. Vectors print as `<1 2 3>`.
`num`	`num(s)`	String to number.
`char`	`char(n)`	ASCII code to single-character string.
`asc`	`asc(s)`	First character to ASCII code.

10.6 File I/O and modules

Function	Signature	Description
`read`	`read(path)`	Read entire file; returns string.
`write`	`write(path, text)`	Write string to file (create or overwrite).
`append`	`append(path, text)`	Append string to file.
`load`	`load(path)`	Execute `.musil` file in current environment.
`eval`	`eval(code)`	Execute string as Musil source code.
`exec`	`exec(cmd)`	Run shell command. Returns exit code.

10.7 Higher-order and control

Function	Signature	Description
`apply`	`apply(f, args)`	Call proc value `f` or proc named `f` (string) with array `args`.
`input`	`input()`	Read one line from stdin.
	`input(prompt)`	Print prompt, then read one line.
`clock`	`clock()`	CPU time in seconds (useful for benchmarking).
`exit`	`exit(code)`	Terminate interpreter with exit code.
`assert`	`assert(cond)`	Throw error if `cond` is falsy.
	`assert(cond, msg)`	Throw error with message.

11 Libraries

Musil ships with a small collection of libraries that extend the core language with I/O, signal processing, scientific computing, and plotting. At the implementation level, these libraries are defined partly as C++ builtins registered in the interpreter, and partly as higher-level Musil procedures written in `.mu` files. The builtins provide efficient access to structured data such as vectors, matrices, WAV files, and plots, while the `.mu` layer offers more convenient composite operations built on top of them.

Each library must be loaded with `load("library.mu")` (the file must be present in the local folder, or installed in `~/.musil/`).

11.1 Standard library

The library is implemented in `stdlib.mu`.

11.1.1 Constants

Name	Value	Description
`PI`	3.14159265358979	π
`TAU`	6.28318530717959	2π
`E`	2.71828182845905	Euler's number e
`PHI`	1.61803398874989	Golden ratio φ
`LN2`	0.693147180559945	$\ln 2$
`SQRT2`	1.41421356237310	$\sqrt{2}$

11.1.2 Core math

Function	Signature	Description
`min`	`min(a, b)`	Smaller of two scalars.
`max`	`max(a, b)`	Larger of two scalars.
`clamp`	`clamp(x, lo, hi)`	Clamp x to $[\texttt{lo}, \texttt{hi}]$. Works on scalars and vectors (via broadcast).
`sign`	`sign(x)`	-1, 0, or 1.
`round`	`round(x)`	Round to nearest integer (half up).
`round_to`	`round_to(x, decimals)`	Round to d decimal places.
`mod`	`mod(a, b)`	Modulo, always non-negative: `mod(-1, 5) = 4`.
`even`	`even(n)`	1 if n is even.
`odd`	`odd(n)`	1 if n is odd.
`between`	`between(x, lo, hi)`	1 if $\texttt{lo} \leq x \leq \texttt{hi}$.
`hypot`	`hypot(a, b)`	$\sqrt{a^2 + b^2}$.
`gcd`	`gcd(a, b)`	Greatest common divisor (integers).
`lcm`	`lcm(a, b)`	Least common multiple.
`ipow`	`ipow(base, exp)`	Integer power ($\texttt{exp} \geq 0$).
`lerp`	`lerp(a, b, t)`	Linear interpolation. Works on scalars and vectors.
`map_range`	`map_range(x,il,ih,ol,oh)`	Re-map from one range to another.
`deg2rad`	`deg2rad(d)`	Degrees to radians.

Function	Signature	Description
`rad2deg`	`rad2deg(r)`	Radians to degrees.

11.1.3 Number formatting

Function	Signature	Description
`pad_left`	`pad_left(s, w, ch)`	Pad string on left to width w with character `ch`.
`pad_right`	`pad_right(s, w, ch)`	Pad string on right to width w.
`fmt_fixed`	`fmt_fixed(x, d)`	Format number with exactly d decimal places.
`fmt_pct`	`fmt_pct(x, d)`	Format as percentage: `fmt_pct(0.752, 1)` → `"75.2%"`.

11.1.4 String utilities

Function	Signature	Description
`starts_with`	`starts_with(s, prefix)`	1 if `s` starts with `prefix`.
`ends_with`	`ends_with(s, suffix)`	1 if `s` ends with `suffix`.
`ltrim`	`ltrim(s)`	Remove leading spaces.
`rtrim`	`rtrim(s)`	Remove trailing spaces.
`trim`	`trim(s)`	Remove leading and trailing spaces.
`repeat_str`	`repeat_str(s, n)`	Repeat string n times.
`count_str`	`count_str(s, pattern)`	Count non-overlapping occurrences of `pattern`.
`replace`	`replace(s, old, new)`	Replace all occurrences of `old` with `new`.

11.1.5 Vector statistics

These functions apply to *vectors*. `mean` and `stdev` also accept arrays (via internal dispatch on `type`).

Function	Signature	Description
`mean`	`mean(x)`	Arithmetic mean. Accepts vector or numeric array.
`stdev`	`stdev(x)`	Population standard deviation. Accepts vector or numeric array.
`normalize`	`normalize(v)`	Scale vector to $[0, 1]$ (min becomes 0, max becomes 1).
`dot`	`dot(a, b)`	Dot product of two vectors: $\sum a_i b_i$.

11.1.6 Array algorithms

Function	Signature	Description
`total`	`total(a)`	Sum of all numeric elements in array. (Note: use `sum` for vectors.)
`minimum`	`minimum(a)`	Smallest element. (Note: `min(a,b)` compares two scalars.)
`maximum`	`maximum(a)`	Largest element.
`contains`	`contains(a, val)`	1 if `val` is in array.
`index_of`	`index_of(a, val)`	First index of `val`, or -1.
`reversed`	`reversed(a)`	New array in reversed order. Original unchanged.
`sorted`	`sorted(a)`	New array sorted ascending (numeric). Original unchanged.
`zip`	`zip(a, b)`	Pair elements: `zip([1,2],["a","b"])` → `[[1,"a"],[2,"b"]]`. Stops at shorter.
`flatten`	`flatten(a)`	Collapse one level of nesting.

11.1.7 Higher-order functions

Function	Signature	Description
`map`	`map(a, f)`	Apply proc `f` to each element; return new array.
`filter`	`filter(a, pred)`	Keep elements for which `pred(x)` is truthy.
`reduce`	`reduce(a, f, init)`	Fold: `acc = f(acc, x)` for each element, starting from `init`.

```
var doubled = map([1,2,3,4], proc(x){ return x * 2 })    # [2,4,6,8]
var evens   = filter([1..8], proc(x){ return mod(x,2)==0 })
var total3  = reduce([1,2,3,4,5], proc(a,x){ return a+x }, 0)    # 15
```

11.1.8 Testing framework

Function	Signature	Description
`assert`	`assert(cond, msg)`	Fail with message if `cond` is falsy.
`assert_eq`	`assert_eq(a, b, msg)`	Fail if `a != b`, showing both values.
`assert_near`	`assert_near(a,b,eps,msg)`	Fail if $\lvert a-b\rvert > \varepsilon$. Use for floating-point tests.
`test_summary`	`test_summary()`	Print pass/fail counts.

Global counters `_tests_run`, `_tests_passed`, and `_tests_failed` are maintained automatically.

11.2 System library

The system library (`system.mu`) provides practical I/O and operating-system oriented services. In the current setup, the most important functions are:

- `sleep(ms)`: suspend execution for a number of milliseconds.

- `clock()`: return the current processor time.
- `dirlist(path)`: list directory contents.
- `filestat(path)`: query basic file information such as existence, size, and permissions.
- `getvar(name)`: retrieve an environment variable.
- `udpsend(host, port, message)` and `udprecv(host, port)`: basic UDP messaging.
- `readcsv(path)` and `writecsv(path, table)`: CSV file import/export.
- `readwav(path)` and `writewav(path, sr, channels)`: WAV file input/output.

At the language level, `readwav` returns a two-element structure `[sr, channels]`, where `sr` is the sample rate and `channels` is an array of vectors, one vector per channel. Conversely, `writewav` expects a filename, a scalar sample rate, and an array of channels of equal length. The underlying implementation supports PCM WAV reading and writing through the companion WAV utilities.

The core library also includes generic text/file helpers such as `read`, `write`, `append`, `load`, `eval`, `exec`, and `apply`. These functions are not part of the separate system header, but they are essential in practice because they make it possible to compose library code, load Musil source files, and build higher-level wrappers in `.mu`.

11.3 Signals library

The signals library (`signals.mu`) is centered on dense numeric vectors and provides synthesis, spectral processing, filtering, convolution, and signal-analysis primitives. The builtins registered by the library include:

- synthesis: `mix`, `gen`, `osc`
- spectral transforms: `fft`, `ifft`, `car2pol`, `pol2car`, `window`
- spectral descriptors: `speccent`, `specspread`, `specskew`, `speckurt`, `specflux`, `specirr`, `specdecr`, `acorrf0`, `energy`, `zcr`
- convolution and channel operations: `conv`, `convmc`, `deinterleave`, `interleave`
- vector/block helpers: `vslice`, `vaddat`
- filters and delay-based effects: `dcblock`, `reson`, `filter`, `filtdesign`, `delay`, `comb`, `allpass`, `resample`

These are the functions explicitly registered by `add_signals(env)`.

Several details are worth stressing. First, the library is intentionally vector-oriented: most functions consume and produce `NumVal` vectors rather than general arrays. FFT-related functions work on interleaved complex data, and the packed representation used by helper functions reduces array traffic by storing paired streams in a single vector when useful. Second, `convmc` is the exception that still uses an array-of-vectors representation, because a multichannel signal is naturally structured as a list of channels rather than a single homogeneous vector. Third, `vaddat` is especially important for overlap-add workflows, because it performs block insertion/addition in compiled code rather than in an interpreted inner loop.

At the `signals.mu` level, one typically defines higher-level DSP procedures by composing the builtins above. In the current Musil setup, these helper procedures include short-time Fourier transform workflows such as `stft` and `istft`, as well as convenience wrappers around `filtdesign` and `filter` for standard biquad types such as low-pass, high-pass, notch, peak, and shelving

filters. These `.mu` procedures do not add new low-level capabilities; rather, they package recurring signal-processing idioms into more ergonomic forms.

11.4 Scientific library

The scientific library (`scientific.mu`) provides matrix algebra, statistics, interpolation, machine learning, and utility procedures for structured numerical work. At the language level, a matrix is represented as an array of numeric row vectors. This representation is deliberately simple and makes expressions such as `M[i]` (row extraction) and `M[i][j]` (element access) natural. The conversion helpers in the implementation explicitly map between this language-level format and the internal C++ matrix class. :contentReference[oaicite:5]index=5

The core matrix and linear-algebra operations include:

- `matdisp`, `matadd`, `matsub`, `matmul`, `hadamard`, `transpose`
- `nrows`, `ncols`, `matsum`, `getrows`, `getcols`
- constructors and structured generators such as `eye`, `rand`, `zeros`, `ones`, `bpf`
- decomposition and algebraic operators such as `inv`, `det`, `diag`, `rank`, `solve`

The library also exposes statistical and analysis-oriented functions such as:

- `median`, `linefit`, `norm`, `dist`
- `matmean`, `matstd`, `cov`, `corr`, `zscore`
- `pca`, `kmeans`, and `knn`

These functions are all implemented as builtins operating on vectors or matrices in the array-of-rows representation.

In practice, the scientific library works especially well together with the plotting layer. Functions such as `matcol`, `getcols`, `linefit`, `pca`, and `kmeans` produce data that can be directly visualized by `plot` and `scatter`. Because matrices remain explicit arrays of row vectors rather than flattened packed buffers, the scientific API is more verbose than the packed signal API, but also more transparent and easier to manipulate interactively.

11.5 Plotting library

The plotting library (`plotting.mu`) provides two builtins:

- `plot(title, y_1, legend_1, y_2, legend_2, ..., style)`
- `scatter(title, x_1, y_1, legend_1, x_2, y_2, legend_2, ..., style)`

In both cases, the last argument is a plotting style:

- `"*"` for line plus markers
- `"."` for markers only
- `"-"` for line only

Both functions generate an SVG file and *return its filename as a string*; they do not print the filename automatically. The SVG backend computes axis ranges, draws simple axes and tick marks, adds optional legends, and stores the file under a sanitized title-based name.

The companion `plotting.mu` layer is meant to collect convenience procedures built on top of the two builtins. Typical helper functions include quick plotting wrappers such as `qplot`, `qplot2`, and `qscatter`, together with column-oriented utilities such as `plot_col`, `plot_cols2`,

and `scatter_cols`. These functions are not performance-critical; their purpose is to reduce repetitive plotting boilerplate in data-analysis scripts.

12 Extending Musil from C++

Musil is designed to be embedded. Any C++ application can register custom builtins that become callable from Musil scripts.

12.1 Including Musil

```cpp
#include "musil.h"

int main() {
    Environment env;
    env.exec("print \"hello\"", "script.mu");
}
```

12.2 Registering a builtin

A builtin has the signature: `Value(std::vector<Value>&, Interpreter&)`.

```cpp
env.register_builtin("midi_to_hz", [](std::vector<Value>& args,
    Interpreter& I) -> Value {
    double midi = std::get<NumVal>(args[0])[0];
    return NumVal{440.0 * std::pow(2.0, (midi - 69.0) / 12.0)};
});
```

After this call, `midi_to_hz(69)` is available in all Musil scripts run through `env`.

12.3 Injecting global variables

```cpp
env.globals["SAMPLE_RATE"] = NumVal{48000.0};
env.globals["SOL_PATH"]    = std::string{"/opt/sol/samples"};
```

12.4 Value types in C++

Musil type	C++ type	Access
number / vector	`NumVal` = `std::valarray<double>`	`std::get<NumVal>(v)`
string	`std::string`	`std::get<std::string>(v)`
array	`ArrayPtr` = `shared_ptr<MusilArray>`	`std::get<ArrayPtr>(v)`
proc	`ProcVal` = `shared_ptr<Proc>`	`std::get<ProcVal>(v)`

Use `std::holds_alternative<T>(v)` to test the type before accessing. Throw `Error{filename, line, message, {}}` for runtime errors from builtins.

13 The Musil IDE

The Musil IDE provides an integrated environment for writing, evaluating, and exploring Musil programs. It is designed to offer a lightweight but expressive workflow, combining a text editor, a console, and interactive tools for inspecting the runtime environment.

13.1 Overview

The IDE is organized into three main areas:

- **Editor**: the main text area where Musil code is written and edited. It supports syntax highlighting, line numbers, and basic editing operations.
- **Console**: displays output produced by the program, including `print` statements and error messages.
- **Listener**: a small input field that allows evaluating single-line expressions interactively, similar to a REPL.
- **Environment Panel**: shows currently defined variables and procedures, allowing quick inspection of the program state.

13.2 Basic Features

The IDE includes the following core features:

- **Run Script**: executes the entire content of the editor.
- **Run Selection**: executes only the selected portion of the code.
- **Stop**: interrupts a running computation (effective between evaluation steps).
- **Reset Environment**: clears all variables and procedures and restarts the Musil environment.
- **Syntax Highlighting**: highlights keywords, strings, comments, and structural elements.
- **Auto-completion**: triggered with `Ctrl+Tab`, suggests builtins and user-defined symbols.
- **Find/Replace**: supports searching and replacing text within the editor.
- **Zoom**: allows increasing or decreasing the font size.
- **Interactive Evaluation**: expressions can be evaluated directly in the listener without modifying the main script.

13.3 File Management

The IDE supports standard file operations:

- Create new files
- Open existing files
- Save and Save As

The current file location is used as a reference for resolving relative paths during execution.

13.4 Libraries and Installation

To access the full capabilities of the language, including higher-level utilities and extended functions, it is necessary to install the Musil standard libraries.

This can be done directly from the IDE:

`Evaluate` → `Install libraries...`

This operation copies the available `.mu` library files into the user directory (typically `~/.musil`), making them available to the `load` function.

Without installing the libraries, only the core built-in functionality of the language is available.

13.5 Notes

- Long computations can be interrupted using the Stop command, but interruption occurs only at safe evaluation points.
- Output is streamed to the console during execution.
- The environment panel updates dynamically after each evaluation.

14 Quick Reference

Language grammar summary

```
statement  ::= var name = expr
             | name = expr
             | name[expr] = expr
             | proc name (params) { stmts }
             | if (expr) { stmts } (else if (expr) { stmts })* (else
                { stmts })?
             | while (expr) { stmts }
             | for (var name in expr) { stmts }
             | return expr
             | break
             | print expr*
             | call

expr       ::= or_expr (< > <= >= == != or_expr)*
or_expr    ::= and_expr (or and_expr)*
and_expr   ::= not_expr (and not_expr)*
not_expr   ::= not not_expr | comparison
comparison ::= add ((< > <= >= == !=) add)*
add        ::= mul ((+ -) mul)*
mul        ::= unary ((* /) unary)*
unary      ::= - unary | atom ([expr])* ((args))?
atom       ::= number | string | [exprs] | (expr) | name | proc(
   params){stmts}
```

Common patterns

```
# Integer loop
for (var i in range(0, 10)) { print i }

# Numeric vector operation
var wave = sin(linspace(0, TAU, 512))

# Map over array
var doubled = map(data, proc(x){ return x * 2 })

# Sort and get min/max
var s = sorted(data)
print s[0]       # minimum
```

```
print s[-1]      # maximum

# Convert array to vector for math
var v = to_vec(numeric_array)
print mean(v) "  std=" stdev(v)

# Filter and reduce
var positives = filter(data, proc(x){ return x > 0 })
var total4    = reduce(positives, proc(a,x){ return a+x }, 0)
```

Licensing

The Musil language is released under the BSD 2-Clause license.